I'm continuing work on my Master's thesis on Federated Learning for LoRaWAN Edge Intelligence.

Context file attached: Thesis-Context-Summary.md
Quick background:
Previous project: Indoor LoRaWAN path loss modeling using centralized ML (XGBoost best model: R^2=0.93). Deployed 6 Arduino MKR WAN 1310 nodes with environmental sensors, trained models offline, created TFLite edge-compatible model + Flutter dashboard.
New thesis upgrade: Add federated learning so devices learn collaboratively without sending raw data—model updates only. Challenge: LoRaWAN has severe communication constraints (duty cycle, ~10 downlinks/day on TTN).
Hardware: Arduino MKR WAN 1310, SCD41/BME280/SPS30 sensors, Kerlink gateway, The Things Network.
Please read the context file first, then help me with: Federated learning in our project as new upgrade ---------- what it is actually give me an really good analogy----

Federated learning (FL) in your LoRaWAN thesis is a way for your 6 Arduino nodes to **learn together** (improve one shared model) without uploading their raw sensor/link datasets—each node trains locally and sends only small "what I learned" updates, which a server aggregates (often by averaging, e.g., FedAvg) and then sends back as an improved global model when downlink budget allows.[1] [2]

A really good analogy (tailored to LoRaWAN)

Imagine your devices are 6 weather-aware "radio scouts" posted in different rooms of a building, and you want them to write one best handbook: *"How to predict path loss / link quality from environment + radio metrics."*[1]

- **Centralized ML (your old project)** is like: every scout photocopies their entire field notebook (raw measurements) and ships it to HQ every day; HQ reads everything and writes the handbook.[1]
- **Federated learning (your upgrade)** is like: each scout *keeps their notebook private* and instead sends HQ only a short "revision note" saying what rules to tweak in the handbook (model weight/gradient updates), and HQ merges all revision notes into the next edition of the handbook.[2] [1]

Now the LoRaWAN twist: your "postal service" is extremely limited—TTN/public LoRaWAN downlinks are scarce (≈10 downlink messages per device per day, and Class A downlinks only happen right after an uplink), so HQ cannot send a full new handbook frequently.[3] [1]

So your FL system behaves like this:

- Scouts **practice locally** using their own recent observations (local training on-device).[2] [1]
- They mail HQ a **tiny summary** of what changed (compressed model update in uplink).[2] [1]
- HQ combines summaries from multiple scouts into a better handbook (aggregation like weighted averaging/FedAvg).[2]
- HQ sends back **rare "edition updates"** (global model updates) only occasionally—daily/weekly, or only to some nodes—because downlink is the bottleneck.[3] [1]

Mapping the analogy to your system

- Scout = Arduino MKR WAN 1310 node learning from its own sensor + RSSI/SNR/SF history.[1]
- HQ/Teacher = your FL orchestrator in the backend that schedules rounds and aggregates updates.[1] [2]
- Handbook = the global TinyML model (TFLite-friendly) used for inference on nodes and updated over time.[1]

If you tell me whether you want the analogy to emphasize **privacy** (no raw data), **communication limits** (downlink scarcity), or **continuous adaptation** (drift over months), I'll tune it into a 2–3 sentence "thesis-ready" version you can paste into your report.

⁂

Federated learning (FL) in your LoRaWAN thesis is a way for your 6 Arduino nodes to learn together (improve one shared model) without uploading their raw sensor/link datasets—each node trains locally and sends only small "what I learned" updates, which a server aggregates (often by averaging, e.g., FedAvg) and then sends back as an improved global model when downlink budget allows ----------make it a really small sentence with a really definitive description.

Federated learning enables your 6 Arduino nodes to collaboratively train one shared model by keeping raw sensor data local and exchanging only compact model updates (gradients/weights) that a central server aggregates, eliminating the need to transmit large datasets while respecting LoRaWAN's severe downlink constraints.

So now let's summarise all our experiments or the project into three distinct main processes. Every node learns from its own sensor, something like that. But give me the specific title for the three main tasks of our project.

Based on your federated learning workflow, here are the three distinct main processes for your project:

1. On-Device Local Training

Each Arduino node independently trains a TinyML model using its own local sensor data (temperature, humidity, CO_2, PM2.5) and link metrics (RSSI, SNR, SF) to predict path loss—without sharing raw measurements [17]

2. Federated Aggregation

The central server (FL orchestrator) collects model updates from participating nodes via LoRaWAN uplinks and aggregates them using FedAvg to create an improved global model [17]

3. Global Model Distribution

The server distributes the updated global model back to nodes via constrained LoRaWAN downlinks (~10/day on TTN), enabling all devices to benefit from collective learning while respecting duty-cycle limits [17]

These three tasks form a continuous cycle: **Learn locally** → **Aggregate centrally** → **Synchronize globally** [17]

⁂

What is the specific use of TinyML in our end device? Is it to philtre out the most relevant data and ignore the data that are not necessary?

TinyML on your end device is mainly used to run a small ML model **locally** (on the Arduino) so the node can predict path loss/link quality in real time and make decisions without relying on the cloud. [18] [19]

What TinyML does on your node

- **On-device inference:** The node uses a TinyML/TFLite-style model to turn current sensor + radio context into an immediate prediction (e.g., expected path loss or link state). [20] [19]
- **Local learning (for FL):** The node can compute small training updates from its recent local samples, then share only those updates (not raw data) during federated rounds. [19] [21]

- **Communication-aware actions:** The prediction can drive behavior like reducing uplinks when conditions are stable and increasing reporting when the link is degrading.[19]

Is it "filtering data"?

Not exactly—"filtering out unnecessary data" is a *possible effect*, but TinyML's core role is prediction/decision-making on-device under tight power/memory/network constraints.[18] [19] That said, TinyML can enable **smart data reduction** (event-triggered uplinks, anomaly/change detection, sending summaries/features instead of raw streams) so you transmit less over LoRaWAN.[22] [19]

Do you want TinyML in your thesis framed primarily as **(A) real-time link prediction**, **(B) uplink reduction via event-triggering**, or **(C) enabling federated local training**?

⁂

Do you want TinyML in your thesis framed primarily as (A) real-time link prediction, (B) uplink reduction via event-triggering, or (C) enabling federated local training?

Here I'm not really sure why I should be using TinyML. What specific purpose? As you know about my past project, what I did and everything. I want you to analyse it really well and give me the best primary function of TinyML in our node. Also, I'm uploading all my PDFs and all the documents from my past projects, so you need to analyse those as well, so that you can give me a better idea about the primary function.

So, I'll also be uploading the files of my past project to make it more relevant, and you can suggest me the best primary function of TinyML in a node.

The best primary function of TinyML on your LoRaWAN node is **real-time on-device inference**: a small path-loss/link-quality model runs locally so the node can make immediate communication decisions (when to transmit, when to stay silent, what to report) without waiting for the cloud.[35] [36]

Why this is the "best" primary purpose (for your thesis)

In your past project, you already proved the value of a lightweight neural network + TensorFlow Lite for real-time, resource-efficient path-loss prediction as a deployable model (derived via pseudo-labeling from a stronger XGBoost model).[35]
Your thesis upgrade adds federated learning, but FL only makes sense if each node can actually *use* (and slightly update) a model locally—TinyML is what makes that feasible on MKR WAN 1310–class constraints.[36] [35]

What it enables on the node (concretely)

- Predict "link health / expected path loss now" from local sensor + radio context, then act immediately (e.g., transmit only on change, raise reporting when degradation is predicted, reduce uplinks when stable).[36]
- Reduce unnecessary LoRaWAN traffic and energy use by making transmissions **event-triggered** instead of always periodic, which fits your "communication-aware intelligence" goal.[36]
- Serve as the "client model" for FL rounds: the node computes small updates from its local data, instead of sending raw data streams.[36]

Is it mainly for filtering data?

Not primarily—"filtering" is a side effect.[36]
TinyML's core role is to turn raw measurements into a **decision signal** (predictions) on the microcontroller, and those decisions can *then* reduce data sent over LoRaWAN (because you only uplink when it matters).[35] [36]

If you want a thesis-ready one-liner title for this role, use:
"TinyML provides on-device link/path-loss inference to enable communication-aware sensing and uplink scheduling under LoRaWAN constraints."[35] [36]

⁂

Predict "link health / expected path loss now" from local sensor + radio context, then act immediately (e.g., transmit only on change, raise reporting when degradation is predicted, reduce uplinks when stable).

Reduce unnecessary LoRaWAN traffic and energy use by making transmissions event-triggered instead of always periodic, which fits your "communication-aware intelligence" goal.
Serve as the "client model" for FL rounds: the node computes small updates from its local data, instead of sending raw data streams.
----------So, yeah, these are the upgrades or features that we're going to be having using TinyML at our end nodes. What advantage are we going to be having, especially using this new enable feature in our new project, besides using the primitive one that we did for the last project? .....
And also, we don't need to use the other machine learning model like XGBoost, Multiple Linear Regression, or some stuff like that, right? After using TinyML, we don't really need to use those heavy machine learning models.

The new advantage of TinyML in your *new* project is that it turns each LoRaWAN node from a "data logger" into an **autonomous edge agent** that can (1) predict link/path-loss locally and (2)

participate in federated learning—so the system keeps improving under changing indoor conditions without shipping raw data to the server.[38] [39]

What you gain vs your last project

Continuous adaptation (not "train once")

In the last project, you trained models offline (XGBoost best) and then deployed a lightweight model mainly for efficient inference.[39]
With TinyML + FL, each node can keep learning from its own environment over time and send only small model updates, so the global model can adapt to drift (occupancy, obstacles, air quality changes) during the campaign.[40] [38]

Real-time decisions under weak connectivity

TinyML's biggest practical edge benefit is local processing: it avoids dependence on cloud connectivity and reduces latency/bandwidth needs, which matters a lot in constrained networks like LoRaWAN.[41] [38]
That directly supports your "communication-aware intelligence" idea—event-trigger uplinks when predicted degradation/change happens, and stay quiet when stable.[38]

Communication efficiency and privacy as a side effect

Because inference and local training happen on-device, you can reduce how often you transmit and avoid sending raw sensor streams (only predictions/alerts and FL updates).[41] [38]

Do you still need XGBoost / MLR / RF?

You usually still need them, but **not on the node**.[39] [38]

- **For research + benchmarking:** Your thesis should still compare against classical models (MLR/RF/XGBoost) to show accuracy trade-offs and justify why the TinyML model is "good enough" for deployment.[39]
- **For creating a strong TinyML model:** In your past work, you explicitly used XGBoost as the high-performing "teacher" and then trained a lightweight neural network using pseudo-labeling so it could be deployed via TensorFlow Lite.[39]
- **For deployment:** The node runs the compact TinyML model; heavy models remain offline/server-side for training, evaluation, or distillation—not for real-time MCU inference.[38] [39]

The "best primary function" statement (thesis-ready)

TinyML's primary function in your node is **on-device path-loss/link-quality inference (and lightweight local training) that enables event-driven communication and federated updates under LoRaWAN constraints.**[38] [39]

If you tell me whether your FL target is "path loss regression" or "link state classification (good/degraded/poor)," I'll suggest which one makes the strongest TinyML story on MKR WAN

1310.

⁂

I mean, even after using TinyML, do we still need to use machine learning models such as EXtreme Boosts, Multiple Linear Regression, Random Forests to analyse the path loss or signal reliability, or is it not necessary? Can we do that just by using this TinyML thing?

Yes—you still *use* models like XGBoost/MLR/RF in the thesis, but mainly **offline/server-side** for benchmarking and for creating/validating the TinyML model; on the node, you run only the compact TinyML model.[55] [56] [57]

TinyML doesn't replace "ML"

TinyML is still machine learning—it's the practice of running an ML model on very constrained hardware (microcontrollers) by keeping the model small and efficient, trading off complexity for deployability.[58] [55]
So you're not choosing "TinyML vs ML"; you're choosing "edge-deployable model vs heavy offline model."[55]

What you can do with only TinyML

You *can* predict path loss / link reliability using only the TinyML model on-device, and that's exactly what makes real-time, communication-aware behavior possible under LoRaWAN constraints.[57] [55]
But if your thesis reports only TinyML results, you lose a strong reference point showing whether your edge model is close to an "upper-bound" model trained offline with full resources.[56]

Why keep XGBoost / MLR / RF anyway

In your last project, XGBoost was your best-performing predictor ($R^2 \approx 0.93$) and you used its outputs as pseudo-labels to train a lightweight neural network that could be deployed with TensorFlow Lite—this is essentially a teacher→student compression idea.[59] [56]
Keeping XGBoost (and optionally MLR/RF) as baselines lets you prove: (1) accuracy trade-off, (2) what performance you "give up" to fit on MKR WAN hardware, and (3) that your federated TinyML approach improves over time compared to a fixed model.[56] [57]

The clean setup for your thesis

A strong, simple experiment set is: **Centralized XGBoost (benchmark)** vs **Centralized TinyML (deployable)** vs **Federated TinyML (your contribution)**.[60] [57] [56]
MLR/RF can stay as "extra baselines" if you have time, but the critical story is showing federated TinyML approaching (or narrowing the gap to) the centralized benchmark while staying deployable.[57] [56]

If you tell me whether you're predicting **path loss (regression)** or **link state (classification)** as your main target, I'll propose the exact 3-model comparison wording for your methodology chapter.

⁂

are there any tinyML libraries in Arduino IDE for MKRWAN1300

Yes—there are TinyML options you can use from the Arduino IDE with the MKR WAN 1300/1310 class of boards, but they're not "MKRWAN-specific" (that library is for LoRaWAN connectivity, not ML).[72] [73]

Practical TinyML choices in Arduino IDE

- **TensorFlow Lite for Microcontrollers (TFLite Micro)**: This is the standard TinyML runtime used in many Arduino TinyML examples, where you compile a small model into a C/C++ header (often `model.h`) and run inference on the microcontroller.[74] [75] [76]
- **EloquentTinyML**: An Arduino library that wraps TensorFlow Lite for Microcontrollers to simplify loading a model and running inference from Arduino sketches.[77]
- **EdgeNeuron**: An Arduino-style library described as providing TensorFlow Lite Micro support for TinyML applications (board support varies, so you'd verify MKR WAN compatibility in practice).[78]

What to expect on MKR WAN 1300/1310

MKR WAN 1300/1310 devices are resource-constrained compared to "big" embedded Linux boards, so the usual approach is: train on PC → convert/quantize → embed the model → run inference on-device.[76] [79]

If you tell me whether you want **inference only** (most common) or **on-device training** (much harder on this class of MCU), I can recommend the most realistic library/setup path for your thesis implementation.

⁂

How to install Arduino_TensorFlowLite on MKRWAN1300

To use TensorFlow Lite Micro on an **MKR WAN 1300** in Arduino IDE, you typically install it **manually** (it's no longer reliably available via Library Manager).[83] [84]

1) Install the MKR WAN 1300 board core

1. Arduino IDE → **Tools** → **Board** → **Boards Manager**.[85]
2. Search "**Arduino SAMD Boards (32-bits ARM Cortex-M0+)**" and install it.[85]
3. Select **Tools** → **Board** → **Arduino SAMD Boards** → **MKR WAN 1300**.[85]

2) Install Arduino_TensorFlowLite (manual)

Option A (recommended): Git clone

1. Find your Arduino libraries folder (typical paths): `Documents/Arduino/libraries` (Windows/macOS) or `~/Arduino/libraries` (Linux).[86]
2. Open a terminal in that folder and run:[86]

```
git clone https://github.com/tensorflow/tflite-micro-arduino-examples Arduino_Tenso
```

3. Restart Arduino IDE, then check **File** → **Examples** → **Arduino_TensorFlowLite** appears.[86]

Option B: Download ZIP (no Git)

Download the repository as a ZIP, extract it, and place/rename the extracted folder to exactly `Arduino_TensorFlowLite` inside your Arduino `libraries` directory, then restart the IDE.[87] [86]

3) Compile an example (sanity check)

Open **File** → **Examples** → **Arduino_TensorFlowLite** → **…** (e.g., **hello_world**) and compile for MKR WAN 1300.[86]

Important note (MKR WAN memory)

The MKR WAN-class nodes you used have very limited MCU RAM/Flash (your report mentions 256 kB Flash and 32 kB SRAM), so only **very small** models will fit, and you'll likely need aggressive model simplification/quantization and a carefully sized tensor arena.[88]

If you tell me your Arduino IDE version (1.8.x vs 2.x) and your OS (Windows/macOS/Linux), I can give you the exact folder path and the most common compile fixes for MKR WAN 1300.

⁂

so my supervisor told me to make a detailed flowchart for our project along with the study of the related papers

You'll want one **high-level flowchart** for the whole system, then smaller sub-flowcharts if needed. For now, let me give you a clear structure so you can draw it (in draw.io, PowerPoint, LaTeX TikZ, etc.) and link it to related FL/TinyML papers.[100] [101] [102] [103]

1. Top-level project flowchart (blocks you should include)

Use this as the **main figure** in the thesis:

1. **Node sensing + local inference (TinyML)**
   - Block: "LoRaWAN End Device (MKR WAN 1310)"
   - Inside:
     - Read sensors (SCD41, BME280, SPS30).[103]
     - Read radio metrics (RSSI, SNR, SF).[103]
     - Run TinyML model → predict path loss / link state.[103]
     - Decide: normal vs degraded link → choose sampling/uplink rate.[103]
2. **Uplink to backend**
   - Block: "LoRaWAN Network (TTN + Gateway)"
   - Arrows: Node → Kerlink gateway → TTN → MQTT/DB.[103]
3. **Federated learning server**
   - Block: "FL Orchestrator (Backend)"
   - Inside:
     - Schedule FL round.[101] [100]
     - Receive local model updates from nodes (uplink).[103]
     - Aggregate (FedAvg) → new global model.[100] [101]
     - Log round info (model version, participating nodes, comm cost).[103]
4. **Downlink global model / metadata**
   - Block: "Model Distribution over LoRaWAN"
   - Inside:
     - Compress/quantize model or send minimal parameters.[104] [103]
     - Respect TTN ~10 downlinks/day and duty cycle.[103]
     - Nodes update local TinyML model when they receive new version.[103]
5. **Dashboard + evaluation**
   - Block: "Monitoring & Evaluation (Flutter Dashboard + DB)"

- Inside:
  - Show model version per node, last update, participation.[103]
  - Compare centralized vs federated model accuracy, comm cost, energy.[103]

Arrange these left-to-right:

`End Devices (TinyML) → LoRaWAN/TTN → FL Orchestrator → LoRaWAN Downlink → Updated End Devices → Dashboard/Evaluation`

2. More detailed flow inside a node (sub-flowchart)

You can add a second flowchart just for **one MKR WAN node**:

1. Start
2. Read sensors + radio metrics.[103]
3. Run TinyML model (TFLite Micro) → predict link quality/path loss.[105] [103]
4. Decision: "Is change significant / degradation predicted?"
   - If **No** → low-rate periodic uplink or skip.[103]
   - If **Yes** → send uplink now (event-trigger).[103]
5. If FL round active:
   - Compute local update from buffered samples.[106] [103]
   - Send compressed update to server via normal uplink.[103]
6. When downlink arrives with new model/params:
   - Update local model/version.[103]
7. Loop.

3. How to tie it to "study of related papers"

When you describe the flowchart in the text, you can map parts to literature:

- FL architecture and round-based loop → generic FL references.[101] [100]
- FL + TinyML on IoT edge devices → the FL+TinyML survey.[107] [102]
- Communication-efficient FL for constrained IoT/LoRa-like links → adaptive/edge FL papers.[108] [107]
- TinyML for on-device inference on MCUs → TinyML overview papers.[104] [105]

If you want, next step I can:

- Turn this into **exact box labels and arrow texts** you can paste into draw.io, or
- Split it into 3 named diagrams: "System architecture", "FL training loop", and "Node decision logic".

⁂

for first lets focus on : 1. Node sensing + local inference (TinyML)

Here is a clean, thesis-ready **flowchart description** for "Node sensing + local inference (TinyML)" that you can directly translate into a diagram.

Title for the diagram

"On-Device Sensing and TinyML Inference at a LoRaWAN End Node" [119]

Boxes and arrows (in order)

1. **Start / Timer Interrupt**
   - Text inside: "Periodic or event-based wake-up (e.g. every 60 s)." [120]
2. **Read Environmental Sensors**
   - Text inside:
     - "Read SCD41: temperature, humidity, CO_2"
     - "Read BME280: pressure"
     - "Read SPS30: PM2.5" [119] [120]
3. **Read Radio Context**
   - Text inside:
     - "Get recent RSSI, SNR, SF, distance index, walls metadata (from config or last packets)." [120] [119]
4. **Preprocess Features**
   - Text inside:
     - "Normalize / scale features using stored parameters (same as in training)."
     - "Assemble feature vector: [RSSI, SNR, SF, temp, humidity, CO_2, PM2.5, pressure, …]." [119] [120]
5. **Run TinyML Model (TFLite Micro)**
   - Text inside:
     - "Load TinyML model (TFLite Micro)."
     - "Inference → predicted path loss / link quality class." [120] [119]
6. **Decision: Link Stable or Degrading?**
   - Diamond, with condition like:
     - "Is |prediction − previous_prediction| > threshold OR is link classified as 'degraded'?" [119]
   - **If NO (link stable):**
     - Arrow to box: "**Low-Rate Reporting / Skip Uplink**"

        - Text: "Optionally send infrequent status packet; otherwise go back to sleep."[119]
    - Then arrow to "**Sleep / Low-Power Mode**" → back to **Start**.
  - **If YES (change or degradation):**
    - Arrow to box: "**Prepare Uplink Payload**"
      - Text: "Pack sensor data + predicted path loss / link state into 18-byte payload."[120] [119]
    - Next box: "**Transmit via LoRaWAN (Class A)**"
      - Text: "Send uplink to gateway/TTN; open RX1/RX2 for possible downlink."[119] [120]
    - Then arrow to "**Sleep / Low-Power Mode**" → back to **Start**.

Optionally, add a small side box from "Run TinyML Model" labeled "**Log (for local FL training buffer)**" if you want to show that predictions + features are stored for later local update computation.[119]

If you want, next I can give you exact draw.io style (swimlane layout, colors, and concise labels) so you can reproduce this diagram quickly for your supervisor.

⁂

how can the end device can itself detect these metrics (RSSI, SNR, SF) without uplink

---

An end device can only "see" **its own transmit settings** (like SF) locally; **RSSI and SNR for the uplink live at the receiver (gateway), not in the transmitting node**, so the node cannot measure *uplink* RSSI/SNR without feedback.[121] [122]

What the node can know by itself

- **Spreading Factor (SF):**
  The node knows the SF it uses to transmit because you configure it in the radio/LoRaWAN stack (e.g., SF7–SF12, ADR off in your experiments).[122] [123]
- **Downlink RSSI/SNR (when it receives something):**
  When the node receives a **downlink** (e.g., join-accept, MAC command, or app downlink), the SX1276 radio can report RSSI/SNR for that *received* packet via API calls or registers (e.g., through LMIC/RadioLib-style functions like `packetRssi()` / `packetSnr()`).[124] [125] [121]
  That gives *downlink* link quality samples, not uplink.

What it cannot know alone

- The node cannot directly measure **uplink RSSI/SNR at the gateway**, because those are measured by the gateway's receiver.[121]
- To use "uplink-quality" metrics onboard, you'd need the **network server/gateway to send them back** in occasional downlinks (e.g., RSSI/SNR summary, link quality flags), and then store them on the node.[121]

Practical implication for your TinyML features

For your **on-node TinyML model**, realistic radio-context inputs are:

- Configured SF, TX power, data rate (known locally).[123] [122]
- Optionally: downlink RSSI/SNR samples from recent downlinks, if you embed them in MAC/app payload and cache them.[121]

If you want, I can propose two versions of your feature set:

1. **"Pure local"** (only what the node truly knows), and
2. **"With occasional feedback"** (gateway sends compressed link metrics back).

⁂

If you want, I can propose two versions of your feature set:

"Pure local" (only what the node truly knows), and
"With occasional feedback" (gateway sends compressed link metrics back).

For your flowchart and model design, these two feature-set versions are realistic on MKR WAN 1310.

1) "Pure local" feature set (no feedback)

These use **only data the node truly has by itself**:

- **Environmental features** (from sensors)
  - Temperature (SCD41)
  - Humidity (SCD41)
  - CO_2 concentration (SCD41)
  - Barometric pressure (BME280)
  - PM2.5 (SPS30)[137] [138]
- **Radio configuration features** (known at transmit time)
  - Spreading Factor used for uplink (SF7–SF12; ADR off or current SF if ADR on)[139] [137]
  - TX power setting (dBm)

    - Uplink interval / current sampling mode (e.g., normal vs event-triggered)[138]
- **Local "history" features** (computed on node)
    - Time since last successful downlink
    - Moving average of recent TinyML predictions (e.g., last N predicted path-loss values or link states)
    - Simple counters: number of missed ACKs/expected downlinks in last window (if you use confirmed uplinks occasionally)

This version is FL- and TinyML-friendly and **does not depend on any gateway data**, which is good under very tight downlink budgets.

2) "With occasional feedback" feature set

Here, the **gateway/network server periodically sends compressed link metrics back** (e.g., once or a few times per day), and the node stores them as extra features.

Additional features you can then use:

- **Uplink quality summaries (from gateway via downlink)**
    - Last reported average uplink RSSI (e.g., over last 50 packets)
    - Last reported average uplink SNR
    - Packet delivery ratio / estimated uplink reliability (e.g., success rate over last window)[140] [141]
- **Network-level hints**
    - Network-suggested "link state" flag (good / marginal / poor), computed server-side
    - Recommended SF or data rate (e.g., ADR suggestion) mapped to a numeric code[142] [139]

These values can be bundled into a small downlink payload (few bytes) and cached in node variables, then used as inputs to the TinyML model alongside the pure-local features. This version gives you **much richer radio-context features** at the cost of rare, carefully scheduled downlinks (which you already need for FL model updates).[138]

If you tell me whether your main prediction target is **path loss (regression)** or **link state (classification)**, I can refine each list to a minimal 6–8 features for a very small TinyML model.

⁂

AS LONG AS THE DOWNLINK RATE IS 10/DAY… DO YOU REALLY THINK THE 10 DATA FOR RSSI ,SNR , PACKET DELIVERY RATIO/ DAY IS ENOUGH FOR OUR TINYml?

Yes, it can still be useful, but you should treat those 10 downlink-based samples per day as **slow, coarse feedback**, not as the main fuel for your TinyML model.

1) What 10 feedback points/day can and cannot do

- **Not enough for training from scratch:**
  10 tuples of "uplink RSSI, SNR, PDR summary" per day per node are far too few to *train* or adapt a model in any meaningful statistical way by themselves.[143] [144]
- **Enough for slow calibration / sanity checks:**
  You can use those downlink metrics to (a) validate whether your locally trained TinyML predictions are roughly in the right range, and (b) adjust **high-level thresholds** (e.g., what you call "good / degraded / poor") occasionally.[145] [146]

2) Where your TinyML should really get its signal from

Given TTN's limit of **10 downlinks/day including ACKs**:[144] [143]

- The **primary features** for TinyML should be what you can sample *freely*: local environment + local configuration + local history (your "pure local" set).[146] [147]
- The **feedback-based features** (average RSSI/SNR/PDR returned once or a few times per day) should be treated as **auxiliary, slowly changing context variables**, not as high-frequency labels.[148] [143]

So in short:

- 10 feedback points/day are **too few to rely on as the main training signal**, but
- they are **enough to (i) sanity-check your model, (ii) slowly tune thresholds, and (iii) enrich features with a "network view" snapshot.**

For your thesis, I'd phrase it like this: the TinyML model is driven primarily by high-rate local sensing, while sparse downlink metrics are used only for coarse calibration and evaluation under TTN's strict 10-downlink/day constraint.[143] [146]

⁂

Yes, it can still be useful, but you should treat those 10 downlink-based samples per day as slow, coarse feedback, not as the main fuel for your TinyML model.

whats coarse feedback?

"Coarse feedback" just means **rough, low-detail, and infrequent feedback**, not a rich, continuous signal.

In your context:

- You only get at most ~10 downlinks per day on TTN.
- Each downlink can carry something like an **average RSSI**, **average SNR**, or **packet delivery ratio over many packets**, not per-packet values.
- So the node sees: "Today your average RSSI was -97 dBm and PDR was 92%" rather than "here is RSSI and SNR for every single uplink."

That's coarse (rough, aggregated) feedback, compared to **fine-grained** feedback where you would see per-packet RSSI/SNR for thousands of transmissions and could train a detailed model directly from that.

⁂

so as long as we had 1,328,334 rows of data for our last project , will the new implementatio of federated learning will be better in terms of correctnes for pathloss prediction in long term?

It can be better in the long term, but **not automatically just because you had 1,328,334 rows before**—it depends on how you use that dataset and design the FL setup.[170] [171]

What your big old dataset gives you

- You already have a very strong centralized model (XGBoost, $R^2 \approx 0.93$) trained on 1.3M rows; this is an excellent **upper bound / teacher** to:
  - Pretrain or distill your TinyML model.
  - Serve as a benchmark to compare federated models against.[171]

When federated learning can be *more correct* long-term

FL can outperform a single "train-once" centralized model **over time** if:[172] [170]

- The environment **drifts** (different semester, different occupancy, furniture changes, HVAC patterns, etc.), so the old 1.3M-row model becomes slightly stale.[170] [171]
- Each node **keeps learning** from its fresh local data and sends updates, so the global model gradually tracks these changes instead of staying frozen.[172] [170]

In that situation, even if the initial centralized model was slightly better at $t = 0$, the **federated, continuously updated TinyML model can be more accurate at $t = 3$–6 months** in real deployment.

When centralized may still win

If conditions stay almost identical to your original campaign and you do not collect much truly new data, then the big centralized model (trained on 1.3M rows) will likely remain the most accurate.[171]
Your FL model may end up approximating it, but not strictly "beating" it in raw accuracy.

Good thesis framing

- Use the 1.3M-row centralized model as **teacher** + **baseline**.[171]
- Show that federated TinyML:
  - Achieves "close-enough" accuracy initially, and
  - Maintains or improves accuracy under environmental changes, **with far less communication and without raw data uploads**.[170] [172]

So: FL is not guaranteed to be more correct from day one, but it is **more robust and adaptable in the long term**, which is exactly what your thesis should argue and evaluate.

⁂

as the federated learning is using the coarse feedback of SNR , RSSI will it be the problem for our accuracy ?

No, coarse feedback (RSSI/SNR summaries from 10 downlinks/day) is not a problem for your accuracy—because it's **not used as training labels** in federated learning.

How federated learning works here (accuracy stays intact)

Your FL setup uses **local supervised learning** on each node, not feedback-driven:[174] [175]

1. **Node collects local data continuously:**
   - Environmental sensors + configured SF/TX power + local history.[174]

- The **ground truth** (path loss / link state) comes from **your existing 1.3M-row dataset** or a simple proxy (e.g., "link good" = successful downlink received).[176] [174]

2. **Node trains locally** using its own labeled samples (from buffered data + proxy labels), computes model updates.[174]
3. **Node sends only model updates** (small gradients/weights), not RSSI/SNR feedback.[175] [174]
4. **Server aggregates** into global model → sends back (rare downlink).[174]

The **coarse RSSI/SNR feedback** is only used **optionally** to:

- **Tune decision thresholds** (e.g., what you call "degraded" locally).
- **Validate model predictions** (e.g., "is my predicted path loss close to server's reported average RSSI?").[177]

Why accuracy is protected

- **Main signal:** High-rate local sensors + historical labels (thousands of samples per node over time).[176]
- **Coarse feedback:** Just a rough sanity check (10/day), not the training source.[178] [177]
- **FL strength:** Each node learns from its unique environmental context, server blends them → **better generalization** than single-node training.[175] [174]

So the coarse feedback is **low-resolution input** for decisions, not the core of your ML accuracy.

⁂

The ground truth (path loss / link state) comes from your existing 1.3M-row dataset or a simple proxy (e.g., "link good" = successful downlink received).

so we still need to use the previous datasets?

No, you do not need to use the previous 1.3M-row dataset on the nodes during live deployment—it stays as an offline resource for your thesis.[179]

During live federated learning (new deployment)

The nodes use **simple proxy labels** for ground truth, generated locally in real-time:[180]

- **For "link state" classification** ("good / degraded / poor"):
  - "Good" = successful downlink received in RX1/RX2 window.
  - "Degraded" = uplink sent but no downlink ACK.
  - "Poor" = multiple missed downlinks in a row.
    This is **completely local** and needs no previous data.[181]

- **For path loss regression** (harder):
  - Use a simple formula like "path loss proxy = TX_power – estimated received power" (from downlink RSSI when available).
  - Or use **pseudo-labels** from your pretrained XGBoost model running on the server (node sends features, server predicts label and sends back rarely).[179]

Role of your old 1.3M-row dataset

It's used **offline** for:

- **Pretraining** the initial global TinyML model (distill XGBoost into TFLite).[179]
- **Benchmarking** ("centralized train-once" vs federated).[180] [179]
- **Validation** (split for holdout testing).[179]

Clean live setup (no old data needed on nodes)

1. **Pre-deploy:** Train initial TinyML model offline using old dataset → flash to all nodes.[179]
2. **Live:** Each node generates its own proxy labels from sensor readings + local link success → trains locally → sends updates.[180]
3. **Server:** Aggregates → improves global model → sends back.[180]

This keeps FL **truly federated** (no raw data uploads) and **fully online** (no old dataset dependency during deployment).[182] [180]

Thesis win: You prove the system works end-to-end with proxy labels alone, while old data establishes your accuracy baseline.

⁂

3. How to tie it to "study of related papers"

When you describe the flowchart in the text, you can map parts to literature:
FL architecture and round-based loop → generic FL references.
FL + TinyML on IoT edge devices → the FL+TinyML survey.
Communication-efficient FL for constrained IoT/LoRa-like links → adaptive/edge FL papers.
TinyML for on-device inference on MCUs → TinyML overview papers.
If you want, next step I can:
Turn this into exact box labels and arrow texts you can paste into draw.io, or
Split it into 3 named diagrams: "System architecture", "FL training loop", and "Node decision logic".

can you provide me more specific papers like using aRDUINO END DEVICES FOR FEDERATED LEARNING FOR lpwan _____

Here are **specific papers/projects** using Arduino or very similar MCUs for federated learning in LoRaWAN/LPWAN:

Exact matches (Arduino + FL + TinyML)

1. "TinyML-FederatedLearning" (GitHub + Electronics Paper)[184] [185] [186]
   - **What:** First fully on-device FL on Arduino Nano 33 BLE Sense (Cortex-M4, 256 kB RAM, 1 MB Flash—**similar resources** to your MKR WAN).
   - **Task:** Keyword spotting (KWS) with TensorFlow Lite Micro + serial FL.
   - **Repo:** https://github.com/marcmonfort/TinyML-FederatedLearning
   - **Key insight:** FL rounds every 5–50 epochs; explores memory/bandwidth trade-offs. **Perfect for your methodology.**[186]
2. "Federated Learning framework for LoRaWAN-enabled IIoT" (arXiv:2410.11612, 2024)[187] [188] [189]
   - **What:** FL anomaly detection on **LoRaWAN IIoT prototypes** (microcontroller + LoRaWAN).
   - **Results:** F1 94.77%, accuracy 92.3%—**compares FL vs centralized.**
   - **Your fit:** They analyze **airtime, rounds/epochs, message size** for LoRaWAN constraints. **Ideal for your comm-efficient FL section.**[187]

Close matches (Embedded FL + LoRaWAN/LPWAN)

3. "Embedded federated learning over a LoRa mesh network" (Pervasive and Mobile Computing, 2023)[190] [191]
   - **What:** FL on **microcontroller boards** (similar to Arduino) using **LoRa mesh** (very close to LoRaWAN).
   - **Your fit:** Directly addresses **embedded FL over LoRa constraints**—perfect for your architecture flowchart.[190]
4. "Federated Learning Over LoRa Networks: Simulator Design" (arXiv:2508.10574)[192]

- **What:** FL simulator specifically for **LoRa networks** (LoRaWAN-like), evaluates FEC/airtime for model updates.
- **Your fit:** Quantifies **LoRa airtime limits** for FL—great for your TTN downlink constraints discussion.[192]

5. **"Federated Learning and TinyML on IoT Edge Devices: Survey" (Zenodo, 2025)**[193] [194]
   - **What:** Comprehensive survey on **FL + TinyML on IoT MCUs** (Arduino mentioned).
   - **Your fit: Maps directly to your node flowchart** (on-device inference + local training).[193]

How to tie these to your flowchart

```
Node Sensing + TinyML Inference → [web:95] "Federated Learning and TinyML on IoT Edge De
FL Orchestrator + Aggregation → [web:134] "Embedded FL over LoRa mesh" + [web:138] "FL f
LoRaWAN Downlink Constraints → [web:136] "FL over LoRa networks"
```

Thesis bonus: The Arduino Nano FL repo is **open code**—you could cite it as "inspiration for on-device FL implementation" and even adapt their serial protocol for LoRaWAN uplinks.[184]

Next: Want exact draw.io XML for your flowchart, or **BibTeX citations** for these papers?

⁂

So I want you to keep all these scripts in a format so that I can transfer the query from here to the visual studio so it can understand everything from this crash, so that I don't have to explain it all again. Give me the best script that you can. It's the matter of life and death.

Below is a compact "context script" you can paste into VS Code (or any new chat) so another assistant understands your thesis setup without you re-explaining. Adjust names if needed.

CONTEXT: MASTER'S THESIS – FEDERATED LEARNING FOR LORAWAN EDGE INTELLIGENCE

Student: Pratik Khadka
Location: France
Thesis working title:
"Decentralized Edge Intelligence for LoRaWAN: Federated Learning for Environment-Driven Path Loss and Link Quality Modeling"

1. Previous project (baseline)

Goal: Model LoRaWAN path loss in an indoor office environment using centralized ML, then deploy a lightweight edge model.

- **Deployment:**
  - 6 × Arduino MKR WAN 1310 end devices
  - Sensors: SCD41 (temperature, humidity, CO_2), BME280 (pressure), SPS30 (PM2.5)
  - Gateway: Kerlink Wirnet iFemtoCell (EU868)
  - Network: The Things Network (TTN, Class A)
- **Data:**
  - Indoor office environment (University of Siegen)
  - Environmental parameters: temperature, humidity, CO_2, PM2.5, barometric pressure
  - Link metrics: RSSI, SNR, Spreading Factor (SF)
  - Collected about 1,328,334 rows over ~4 months
- **ML pipeline (centralized, offline):**
  - Models: Multiple Linear Regression, Random Forest, XGBoost
  - Best: XGBoost, $R^2 \approx 0.93$, RMSE ≈ 5.14 dB for path loss prediction
  - Trained a compact neural network using pseudo-labels from XGBoost (teacher–student style)
  - Converted to TensorFlow Lite (TinyML) for edge deployment
- **System pipeline:**
  Gateway → TTN/TTS → Node-RED → MQTT → PostgreSQL → Flutter desktop dashboard
- **Dashboard:**
  Real-time visualization of environmental variables, RSSI, SNR, SF, path-loss predictions.

2. New thesis upgrade: add TinyML + Federated Learning (FL)

High-level idea:
Move from *centralized data collection + offline training* to *distributed on-device learning + federated aggregation* over LoRaWAN, under strict communication constraints.

2.1 Hardware / network (same base)

- End devices: 6 × Arduino MKR WAN 1310 (STM32L0, SX1276)
- Sensors: SCD41, BME280, SPS30
- Gateway: Kerlink Wirnet iFemtoCell (EU868)
- Network: The Things Network (community, Class A, EU868 duty-cycle limits)

2.2 LoRaWAN constraints (very important)

- TTN fair-use: ~10 downlink messages per device per day (includes ACKs)
- Class A: downlinks only possible after uplinks (RX1/RX2 windows)
- Duty cycle: 1% in EU868 band
- This makes "classic FL" (full model to every device each round) **impractical**.

3. Role of TinyML on the node

Primary function of TinyML:
On each MKR WAN node, run a small TFLite Micro/TinyML model **locally** to predict path loss / link quality from local sensor and radio context, and use this prediction to make **communication-aware decisions** (when/how often to send uplinks) and to produce local model updates for federated learning.

TinyML is **not** just filtering data; it enables:

1. Real-time on-device inference (path loss or link state).
2. Event-triggered transmission (reduce uplinks when stable, increase when degradation is predicted).
3. Local training steps that generate FL updates without sending raw data.

We consider two feature-set variants for the TinyML model:

3.1 "Pure local" features (no network feedback)

Only information truly available on the node:

- Environmental: temperature, humidity, CO_2, pressure, PM2.5
- Radio configuration: SF (current spreading factor), TX power, current sampling/uplink mode
- Local history: moving average of recent predictions, time since last downlink, count of missed expected downlinks, etc.

3.2 "With occasional feedback" features (sparse network view)

Because TTN allows only ~10 downlinks/day, we treat gateway feedback as **coarse, low-rate** context, not as main training labels.

Occasional downlinks carry aggregated metrics, e.g.:

- Average uplink RSSI over last N packets
- Average uplink SNR
- Packet delivery ratio estimate (PDR)
- Optional "link state" flags from the server (good / marginal / poor)

- Optional ADR-style SF suggestions

These are used to *calibrate thresholds* and enrich features, not as dense training labels.

4. Node-side flow: "Node sensing + local inference (TinyML)"

This is the first main flowchart the assistant should understand and help refine.

On each MKR WAN 1310 node:

1. **Wake-up (timer or event)**
   - Periodic or event-based (e.g., every 60 s).
2. **Read sensors**
   - SCD41: temperature, humidity, CO_2
   - BME280: pressure
   - SPS30: PM2.5
3. **Read radio context (local)**
   - Current SF, TX power, current operating mode (normal / high-rate).
   - Local history variables (previous predictions, time since last downlink, etc.).
   - Optionally cached feedback: last reported avg RSSI/SNR/PDR from a previous downlink.
4. **Preprocess features**
   - Scale/normalize using stored parameters.
   - Build feature vector (subset of the above).
5. **Run TinyML model (TFLite Micro)**
   - Inference → predicted **path loss** (regression) or **link state** (classification: good / degraded / poor).
6. **Decision logic (communication-aware)**
   - If link is stable / prediction change < threshold → low-rate reporting or skip uplink.
   - If model predicts degradation or large change → send uplink now (event trigger).
7. **Uplink via LoRaWAN (Class A)**
   - Payload includes sensor data and optionally predictions / model metadata.
8. **Optional: FL local training buffer**
   - Store recent samples + proxy labels for local update computation.
9. **Sleep / low-power**
   - Until next wake-up.

5. FL architecture for this project

5.1 Global loop (system-level)

- **FL Orchestrator (backend):**
  - Schedules federated rounds (e.g., daily/weekly due to airtime limits).
  - Receives local model updates from nodes via uplink.
  - Aggregates (e.g., FedAvg) into a new global model.
  - Logs round metadata: model version, participating nodes, communication cost, etc.
  - Compresses/quantizes global model for LoRaWAN-friendly distribution.
- **Model distribution:**
  - Sends updated global model or small deltas/metadata back via **rare downlinks**, respecting ~10 downlinks/day/device and duty cycle.
  - Not all devices must be updated every round.
- **Dashboard:**
  - Shows per-node model version, last round participation, current prediction error, and communication statistics.
  - Compares centralized "train-once" model vs federated continuously updating model.

5.2 Targets and labels in FL

We may choose either:

- **Path loss regression:** predict path loss in dB (using proxies/labels derived from original dataset and/or occasional server feedback).
- **Link state classification:** classify link as {good, degraded, poor}, with local proxy labels based on success/failure of downlinks and PDR.

Important:
During live deployment, **nodes do not use the old 1.3M-row dataset** directly. That dataset is used **offline** to:

- Pretrain a strong centralized model (XGBoost) and distill a TinyML model.
- Provide benchmarks and evaluation baselines.
- Possibly generate initial labels/proxies.

Live FL relies on **local proxy labels** and continuous sensing.

6. TTN / LoRaWAN constraints the solution must respect

- Max ~10 downlinks per device per day (TTN community fair use).
- Class A downlinks only after uplinks (RX1/RX2).
- 1% duty cycle in EU868 band.
- Therefore:
  - FL updates must be infrequent and small.
  - Most intelligence must come from **local TinyML inference** + **local training**, not from frequent model pushes or dense feedback.
  - Coarse feedback on RSSI/SNR/PDR is acceptable as infrequent context, not as main training signal.

7. Key references we want to align with

The assistant should relate our design/flowcharts to these types of works:

- **FL over LoRa / LoRaWAN:**
  - "Federated Learning framework for LoRaWAN-enabled IIoT communication: A case study."
  - "Embedded federated learning over a LoRa mesh network."
  - "Federated Learning Over LoRa Networks: Simulator Design…".
- **TinyML + FL on microcontrollers / Arduino-like boards:**
  - GitHub/works: "TinyML-FederatedLearning" (Arduino Nano 33 BLE Sense + TFLite Micro, on-device FL).
  - Survey: "Federated Learning and TinyML on IoT edge devices."
- **TinyML on MCUs in general:**
  - TinyML overview and on-device inference surveys (TFLite Micro, model compression, knowledge distillation).

8. What I want help with in this new environment

When I paste this context, I want the assistant to be able to:

1. Design and refine **flowcharts** for:
   - Node sensing + TinyML inference (already outlined).
   - FL training loop (client–server).
   - Overall system architecture.
2. Help choose and justify:
   - Feature sets (pure local vs with feedback).
   - Target type (path loss regression vs link state classification).

- FL strategy under strict LoRaWAN constraints (round frequency, update size, client selection).

3. Map each part of the architecture to **specific related papers** and help write thesis-level text.

You are now fully briefed on my previous centralized ML project and the new federated TinyML + LoRaWAN thesis. Use this context for all future answers so I don't need to repeat it.

So I want you also to mention something about what kind of changes it's going to be in our new thesis or new research or new project, despite what it was there regarding our previous project. Mention something about concrete ideas about the Federated Learning on the device, its capabilities, and all. Try pulling every information from the web or from the contents that are available that I uploaded recently and once again, be really, really careful. I want you to be really correct or have high efficiency, because once again it's a matter of life and death.

In your new thesis, the system changes from "smart measurement campaign + offline ML" to a **living, distributed learning system** where each LoRaWAN node runs TinyML, trains locally, and contributes to a global model under strict airtime limits.[200] [201] [202] [203]

1. What was before vs what is new

Previous project (centralized ML, static model)

- All 6 MKR WAN 1310 nodes acted as **data loggers**: they measured environment + RSSI/SNR/SF and pushed everything to the backend.[203]
- You trained MLR, RF, and XGBoost centrally on ~1.3M rows; XGBoost was best ($R^2 \approx 0.93$).[203]
- You distilled a lightweight neural network from XGBoost and deployed it via TFLite mainly for **efficient inference / visualization** on a PC dashboard, not as an autonomous edge learner.[203]
- Model was "**train once, deploy, forget**"; no continual adaptation to drift during deployment.[202] [203]

New thesis (federated TinyML, adaptive model)

- Each MKR WAN 1310 becomes an **intelligent client**: it runs a TinyML model, learns from its own sensor history, and sends only model updates.[204] [205] [202]
- A backend FL orchestrator aggregates updates (FedAvg-style) and periodically pushes a compressed global model back over heavily constrained LoRaWAN downlinks.[201] [202]
- The model is **continuously refined** over days/weeks as conditions change (occupancy, obstacles, air quality), instead of being frozen.[206] [202]
- Communication is **event-driven**: nodes adapt their uplink rate based on predicted link quality rather than fixed intervals, reducing airtime and energy use.[200] [202]

2. Concrete new capabilities on the device

2.1 On-device sensing + TinyML decision making

Each node now:[204] [202] [203]

- Reads SCD41 (temp, humidity, CO_2), BME280 (pressure), SPS30 (PM2.5) plus local radio settings (SF, TX power, sampling mode).
- Runs a TFLite Micro model that predicts either:
  - **Path loss** (regression), or
  - **Link state**: good / degraded / poor (classification).
- Uses predictions to decide:
  - When to transmit (event-trigger vs periodic).
  - Whether to increase/decrease sampling rate.
  - When to flag possible link degradation.

Result: The node is no longer a dumb sensor; it **interprets** its context and **acts** on it in real time, with MCU-level TinyML as described in current TinyML surveys.[207] [208]

2.2 Local training for federated learning

Beyond inference, each node can:[205] [202] [204]

- Keep a short buffer of recent samples and **proxy labels**:
  - For classification: "good link" vs "bad link" from success/failure of downlinks or PDR thresholds.[209] [201]
  - For regression: approximate path-loss targets from TX power and occasional RSSI feedback, or pseudo-labels occasionally provided by a strong server-side model distilled from your 1.3M-row dataset.[210] [203]
- Run a few **local training epochs** (TinyML-compatible, small batch sizes) to update its weights.

- Send only the **parameter deltas or gradients** in uplinks during FL rounds, as in LoRaWAN-FL case studies where model updates instead of raw data dramatically cut bandwidth and preserve privacy.[211] [201]

2.3 Communication-efficient behaviour under TTN limits

Because TTN gives you roughly **10 downlinks/day/device**, the new system:[212] [213] [202]

- Treats uplinks as the main channel for **FL updates and application data**, carefully sized to LoRa payload limits.
- Uses **rare, coarse feedback** downlinks to convey:
  - Aggregated RSSI/SNR/PDR,
  - Global model version, or
  - A small compressed model delta or threshold parameters.[212] [200]
- Schedules FL rounds **infrequently** (e.g., daily/weekly) and only with a subset of nodes to respect airtime budgets, consistent with LoRaWAN-FL experiments that show the need to tune rounds/epochs to airtime.[201] [212]

3. How the research contribution changes

3.1 From "offline modeling" to "online, distributed adaptation"

- Old work: measure → upload everything → train offline → evaluate. Good for understanding indoor behavior, not for long-running autonomous systems.[203]
- New work: the network itself **learns over time**, combining on-device updates with FL aggregation as recommended in FL+TinyML surveys for edge AI.[205] [202] [204]
- This demonstrates **federated continual learning** in a real LPWAN deployment, which current surveys highlight as an open direction.[206] [205]

3.2 From single-site data to multi-client heterogeneity

- Old work: centralized dataset from one campaign, all data pooled.[203]
- New work: each node sees **a different micro-environment** (stairwell, corridor, office, server room, kitchen).[203]
- FL explicitly handles such non-IID, unbalanced data and still reaches accuracy comparable to centralized training, as shown in LoRaWAN FL cases (F1 ≈ 94.8%, accuracy ≈ 92.3% in anomaly detection with uneven clients).[211] [201]

3.3 From raw data upload to privacy- and bandwidth-aware learning

- Old work: all sensor and link data sent to server; privacy and bandwidth considered but not minimized by design.[203]
- New work: **data never leaves the device**; only model updates do, aligning with LoRa-FL security work that uses FL to detect issues while preserving data privacy.[209] [201]

- This is exactly the FL-TinyML direction emphasized in recent frameworks that combine LoRa communication, OTA model updates, and compression.[214] [200]

4. How to express this in your thesis (one strong paragraph)

You can safely write something like:

> "Compared to our previous centralized path-loss modeling campaign—where all 1.3M measurements were uploaded to a server and a single XGBoost model was trained offline—we now implement a federated TinyML architecture in which each MKR WAN 1310 node runs a compact on-device model, learns from its own local sensor and link history, and shares only model updates over LoRaWAN. This design follows recent work on FL and TinyML for IoT edge devices and LoRa-based IIoT, which shows that federated models can match centralized accuracy while drastically reducing bandwidth and preserving data privacy in resource-constrained networks."[215] [202] [204] [200] [201] [203]

If you tell me your *final* choice of target (path-loss regression vs link-state classification), I can turn all this into 3–4 bullet "core contributions" tailored exactly to your thesis abstract and conclusion.

⁂

1. Thesis-Context-Summary.md
2. https://cloud.google.com/discover/what-is-federated-learning
3. https://www.thethingsnetwork.org/forum/t/fair-use-policy-explained/1300
4. https://www.datacamp.com/blog/federated-learning
5. https://apxml.com/courses/federated-learning/chapter-2-advanced-aggregation-algorithms/fednova-algorithm
6. https://www.sciencedirect.com/topics/computer-science/federated-learning
7. https://www.nature.com/articles/s41598-025-24622-3
8. https://www.reddit.com/r/TheThingsNetwork/comments/toyrbq/fair_use_policy/
9. https://www.ibm.com/think/topics/federated-learning
10. https://arxiv.org/html/2311.10291v2
11. https://en.wikipedia.org/wiki/Federated_learning
12. https://www.couchbase.com/blog/federated-learning/
13. https://pmc.ncbi.nlm.nih.gov/articles/PMC11466570/
14. https://research.ibm.com/blog/what-is-federated-learning
15. https://www.shadecoder.com/topics/fedavg-algorithm-a-comprehensive-guide-for-2025
16. https://lora.vsb.cz/index.php/sending-downlinks-via-tts-using-a-python-script/
17. Thesis-Context-Summary.md
18. https://pmc.ncbi.nlm.nih.gov/articles/PMC12115890/
19. Thesis-Context-Summary.md

20. https://thinkrobotics.com/blogs/learn/tinyml-applications-on-microcontrollers-revolutionizing-edge-ai
21. https://cloud.google.com/discover/what-is-federated-learning
22. https://pmc.ncbi.nlm.nih.gov/articles/PMC9227753/
23. https://orbitive.tech/blog/tinyml-at-the-edge-deploying-energy-efficient-anomaly-detection-on-microcontrollers
24. https://www.geeksforgeeks.org/machine-learning/feature-selection-vs-feature-extraction/
25. https://pmc.ncbi.nlm.nih.gov/articles/PMC9962960/
26. https://www.innovatiana.com/en/post/feature-extraction-in-ai
27. https://runtimerec.com/real-time-anomaly-detection-for-predictive-maintenance-using-tinyml/
28. https://www.datacamp.com/tutorial/feature-extraction-machine-learning
29. https://www.datacamp.com/blog/what-is-tinyml-tiny-machine-learning
30. https://www.sciencedirect.com/science/article/pii/S1110016825009433
31. https://talent500.com/blog/what-is-tinyml-introduction/
32. https://cycle.io/learn/tiny-ml-and-on-device-ml
33. https://www.geeksforgeeks.org/machine-learning/what-is-tinyml-tiny-machine-learning/
34. https://thinkrobotics.com/blogs/learn/introduction-to-tinyml-on-microcontrollers-bringing-ai-to-the-edge
35. LoRaWAN-in-Indoor-Office-Environments-12th-July-V5.pdf
36. Thesis-Context-Summary.md
37. LoRAWAN.pdf
38. Thesis-Context-Summary.md
39. LoRaWAN-in-Indoor-Office-Environments-12th-July-V5.pdf
40. https://milvus.io/ai-quick-reference/how-is-federated-learning-implemented-on-edge-devices
41. https://pmc.ncbi.nlm.nih.gov/articles/PMC12115890/
42. https://www.arxiv.org/pdf/2010.08678v1.pdf
43. https://pmc.ncbi.nlm.nih.gov/articles/PMC9227753/
44. https://fiveable.me/edge-ai-and-computing/unit-9/federated-learning-edge-devices/study-guide/wwf5DLVqxmcyX5EB
45. https://stackoverflow.com/questions/63843254/in-knowledge-distillation-how-to-run-the-student-and-the-teacher-models-in-para
46. https://pmc.ncbi.nlm.nih.gov/articles/PMC12722349/
47. https://edgify.ai/2024/06/03/distributed-training-on-edge-devices-large-batch-vs-federated-learning/
48. https://www.silicloud.com/blog/what-is-model-distillation-in-tensorflow-and-how-can-it-be-used-to-compress-models/
49. https://www.datacamp.com/blog/what-is-tinyml-tiny-machine-learning
50. https://www.sciencedirect.com/science/article/pii/S0029801825021924
51. https://pub.towardsai.net/adaptive-multi-teacher-distillation-for-enhanced-supervised-learning-e70062acce7e

52. https://www.kaggle.com/code/colearninglounge/in-depth-model-building-and-training-with-ml-clas
53. https://ieeexplore.ieee.org/iel8/6287639/10820123/11266886.pdf
54. https://markaicode.com/knowledge-distillation-lightweight-models/
55. https://pmc.ncbi.nlm.nih.gov/articles/PMC12115890/
56. LoRaWAN-in-Indoor-Office-Environments-12th-July-V5.pdf
57. Thesis-Context-Summary.md
58. https://cycle.io/learn/tiny-ml-and-on-device-ml
59. https://www.tencentcloud.com/techpedia/121607
60. https://milvus.io/ai-quick-reference/how-is-federated-learning-implemented-on-edge-devices
61. https://pmc.ncbi.nlm.nih.gov/articles/PMC9227753/
62. https://arxiv.org/abs/2506.14440
63. https://www.nature.com/articles/s41598-025-23398-w
64. https://arxiv.org/pdf/2210.16556.pdf
65. https://fiveable.me/edge-ai-and-computing/unit-5/knowledge-distillation/study-guide/2oERz1yDJ4t4fKW7
66. https://blog.meetneura.ai/federated-learning-edge-devices/
67. https://www.ijprems.com/uploadedfiles/paper/issue_10_october_2025/44164/final/fin_ijprems1760279127.pdf
68. https://arxiv.org/abs/2406.03409
69. https://talent500.com/blog/what-is-tinyml-introduction/
70. https://www.sciencedirect.com/science/article/pii/S2590005625003017
71. https://www.embedded.com/deploying-neural-networks-on-microcontrollers-with-tinyml/
72. https://github.com/arduino-libraries/MKRWAN_v2
73. https://github.com/arduino-libraries/MKRWAN
74. https://docs.arduino.cc/tutorials/nano-33-ble-sense/get-started-with-machine-learning/
75. https://cms.tinyml.org/wp-content/uploads/emea2021/tinyML_Talks_Massimo_Banzi_210601.pdf
76. https://cms.tinyml.org/wp-content/uploads/2020/09/bay-area-situnayake.pdf
77. https://github.com/eloquentarduino/EloquentTinyML
78. https://docs.arduino.cc/libraries/edgeneuron/
79. https://pmc.ncbi.nlm.nih.gov/articles/PMC12115890/
80. https://docs.arduino.cc/libraries/harvard_tinymlx/
81. https://docs.arduino.cc/libraries/arduino_tinygl/
82. https://www.arduinolibraries.info/libraries
83. https://forum.arduino.cc/t/why-i-cant-find-the-arduino-tensorflowlite-library-in-library-management/1044070
84. https://github.com/arduino/ArduinoTensorFlowLiteTutorials/issues/32
85. https://docs.arduino.cc/software/ide-v1/tutorials/getting-started/cores/arduino-samd
86. https://github.com/tensorflow/tflite-micro-arduino-examples

87. https://discuss.ai.google.dev/t/not-able-to-find-arduino-tensorflowlite-library-in-arduino-ide/24658
88. LoRaWAN-in-Indoor-Office-Environments-12th-July-V5.pdf
89. https://forum.arduino.cc/t/tensorflow-for-arduino-nano-33-ble-sense-rev-2/1136022
90. https://github.com/spaziochirale/Chirale_TensorFlowLite
91. https://fdmcs.math.cnrs.fr/gerardo.granados/tensorflow/-/blob/1923123d32ea41d92b70a27a3f6ecf0763b56f6c/tensorflow/lite/micro/examples/person_detection/README.md
92. https://www.reddit.com/r/esp32/comments/1eciw9m/does_tensorflowlite_esp32_library_work_in_arduino/
93. https://sming.readthedocs.io/en/5.1.0/_inc/Sming/Libraries/Arduino_TensorFlowLite/index.html
94. https://www.arduinolibraries.info/libraries/ardu-tf-lite
95. https://blog.tensorflow.org/2019/11/how-to-get-started-with-machine.html
96. https://docs.arduino.cc/libraries/microtflite/
97. Thesis-Context-Summary.md
98. https://wiki.seeedstudio.com/Wio-Terminal-TinyML-TFLM-1/
99. https://github.com/spaziochirale/ArduTFLite
100. https://docs.nvidia.com/clara/clara-train-archive/3.1/federated-learning/fl_background_and_arch.html
101. https://www.sciencedirect.com/science/article/pii/S2405844024141680
102. https://www.sciencedirect.com/science/article/pii/S2405959525000839
103. Thesis-Context-Summary.md
104. https://pmc.ncbi.nlm.nih.gov/articles/PMC9227753/
105. https://pmc.ncbi.nlm.nih.gov/articles/PMC12115890/
106. https://milvus.io/ai-quick-reference/how-is-federated-learning-implemented-on-edge-devices
107. https://pmc.ncbi.nlm.nih.gov/articles/PMC11579390/
108. https://www.nature.com/articles/s41598-024-78239-z
109. https://towardsdatascience.com/practical-federated-learning-with-azure-machine-learning-8807f9bd1a7e/
110. https://arxiv.org/html/2502.05273v1
111. https://nvflare.readthedocs.io/en/2.6/fl_introduction.html
112. https://apxml.com/courses/federated-learning/chapter-6-federated-learning-system-design/fl-system-architecture
113. https://www.bohrium.com/paper-details/federated-learning-and-tinyml-on-iot-edge-devices-challenges-advances-and-future-directions/1148657804865699889-2702
114. https://arxiv.org/html/2407.20573v2
115. https://www.francescatabor.com/articles/2025/5/3/technical-architecture-of-federated-learning
116. https://arxiv.org/html/2412.01609v1
117. https://magnet.gitlabpages.inria.fr/declearn/docs/2.1/user-guide/fl_process/
118. https://smartcity.efri.uniri.hr/article.php?id=26328
119. Thesis-Context-Summary.md

120. LoRaWAN-in-Indoor-Office-Environments-12th-July-V5.pdf
121. https://www.thethingsnetwork.org/docs/lorawan/rssi-and-snr/
122. https://www.thethingsnetwork.org/docs/lorawan/spreading-factors/
123. LoRaWAN-in-Indoor-Office-Environments-12th-July-V5.pdf
124. https://forum.arduino.cc/t/read-out-rssi-connection-quality/1128226
125. http://community.heltec.cn/t/how-to-read-rssi-snr-with-heltecs-lorawan-library/3762
126. https://github.com/jgromes/RadioLib/discussions/1022
127. https://www.lab5e.com/docs/lora/dr_sf/
128. https://docs.thingpark.com/thingpark-enterprise/latest/docs/user-guide/network-tools/wireless-logger/lorawan-traffic/
129. https://www.thethingsnetwork.org/forum/t/poor-rssi-in-sx1276/12478
130. https://eicut.com/guide-to-spreading-factors-and-adr-in-lorawan/
131. https://www.thethingsnetwork.org/forum/t/tinylora-raspberry-node-how-can-i-display-the-rssi/26684
132. https://forum.lora-developers.semtech.com/t/very-low-rssi-values-measured-at-rx-sx1276/841
133. https://community.element14.com/challenges-projects/project14/remote-monitoring-control-devices/b/blog/posts/lora-experimental-environmental-sensors-loraxes---initial-rssi-tests
134. https://sensing-labs.com/f-a-q/a-good-radio-level/
135. https://github.com/jgromes/RadioLib/issues/205
136. https://www.reddit.com/r/embedded/comments/1h1ykyt/lora_module_rssi_mapping/
137. LoRaWAN-in-Indoor-Office-Environments-12th-July-V5.pdf
138. Thesis-Context-Summary.md
139. https://www.thethingsnetwork.org/docs/lorawan/spreading-factors/
140. https://www.thethingsnetwork.org/docs/lorawan/rssi-and-snr/
141. https://sensing-labs.com/f-a-q/a-good-radio-level/
142. https://eicut.com/guide-to-spreading-factors-and-adr-in-lorawan/
143. https://www.thethingsnetwork.org/forum/t/fair-use-policy-explained/1300
144. https://www.youtube.com/watch?v=G3GKT2HQQF8
145. https://pmc.ncbi.nlm.nih.gov/articles/PMC11579390/
146. Thesis-Context-Summary.md
147. LoRaWAN-in-Indoor-Office-Environments-12th-July-V5.pdf
148. https://www.thethingsnetwork.org/docs/lorawan/rssi-and-snr/
149. https://redmine.auroville.org.in/issues/8299
150. https://arxiv.org/html/2405.07601v2
151. https://arxiv.org/pdf/2103.08295.pdf
152. https://www.sciencedirect.com/science/article/abs/pii/S2542660522001238
153. https://things4891.rssing.com/chan-56144148/article91197.html

154. https://jmsr-online.com/article/practical-tinyml-hands-on-model-training-and-deployment-378/
155. https://arxiv.org/abs/2110.11775
156. https://core-electronics.com.au/guides/send-a-downlink-from-adafruit-io-via-the-things-network/
157. https://dl.acm.org/doi/10.1145/3776588
158. https://www.thethingsnetwork.org/forum/t/can-i-avoid-fair-access-policy-by-implementing-a-private-ttn-network/2986
159. https://www.reddit.com/r/TheThingsNetwork/comments/toyrbq/fair_use_policy/
160. https://github.com/JohanScheepers/ttn-sandbox-lmic-node
161. https://www.reddit.com/r/LanguageTechnology/comments/bh63e8/what_does_coarsegrained_mean_in_machine_learning/
162. https://openreview.net/forum?id=vLqkCvjHRD
163. https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall
164. https://www.sciencedirect.com/science/article/pii/S2666920X24001358
165. https://afse2017.sciencesconf.org/143465/estimating_choice_parameters.pdf
166. https://www.math.uwaterloo.ca/~rneogi/comb-opt-group/talk/learning-from-coarse-samples/
167. https://www.sciencedirect.com/science/article/abs/pii/S0020025522009343
168. https://arxiv.org/abs/2305.18341v2
169. https://arxiv.org/abs/2406.16807
170. Thesis-Context-Summary.md
171. LoRaWAN-in-Indoor-Office-Environments-12th-July-V5.pdf
172. https://pmc.ncbi.nlm.nih.gov/articles/PMC11579390/
173. image.jpg
174. Thesis-Context-Summary.md
175. https://pmc.ncbi.nlm.nih.gov/articles/PMC11579390/
176. LoRaWAN-in-Indoor-Office-Environments-12th-July-V5.pdf
177. https://www.thethingsnetwork.org/docs/lorawan/rssi-and-snr/
178. https://www.thethingsnetwork.org/forum/t/fair-use-policy-explained/1300
179. LoRaWAN-in-Indoor-Office-Environments-12th-July-V5.pdf
180. Thesis-Context-Summary.md
181. https://www.thethingsnetwork.org/docs/lorawan/rssi-and-snr/
182. https://pmc.ncbi.nlm.nih.gov/articles/PMC11579390/
183. image.jpg
184. https://github.com/marcmonfort/TinyML-FederatedLearning
185. https://github.com/NilLlisterri/TinyML-FederatedLearning
186. https://marcmonfort.com/posts/10ac8a53/
187. https://arxiv.org/html/2410.11612v1

188. https://arxiv.org/html/2410.11612
189. https://arxiv.org/abs/2410.11612
190. https://www.sciencedirect.com/science/article/pii/S1574119223000779
191. https://dl.acm.org/doi/10.1016/j.pmcj.2023.101819
192. https://arxiv.org/pdf/2508.10574.pdf
193. https://www.sciencedirect.com/science/article/pii/S2405959525000839
194. Thesis-Context-Summary.md
195. https://www.reddit.com/r/Lora/comments/1gtc8s5/need_help_with_the_lorawan_data_transmission_and/
196. https://paperswithcode.com/paper/federated-learning-framework-for-lorawan
197. https://marcmonfort.com/posts/6a281998/
198. https://pmc.ncbi.nlm.nih.gov/articles/PMC8347601/
199. https://dl.acm.org/doi/10.1145/3462203.3475896
200. https://nchr.elsevierpure.com/en/publications/federated-learning-and-tinyml-on-iot-edge-devices-challenges-adva/
201. http://arxiv.org/abs/2410.11612
202. Thesis-Context-Summary.md
203. LoRaWAN-in-Indoor-Office-Environments-12th-July-V5.pdf
204. https://www.sciencedirect.com/science/article/pii/S2405959525000839
205. http://www.puiij.com/index.php/research/article/view/178
206. https://labs.sciety.org/articles/by?article_doi=10.20944%2Fpreprints202512.0118.v1
207. https://arxiv.org/html/2506.18927v1
208. https://pmc.ncbi.nlm.nih.gov/articles/PMC9227753/
209. https://pmc.ncbi.nlm.nih.gov/articles/PMC11597939/
210. https://www.sciencedirect.com/science/article/pii/S1566253523005055
211. https://arxiv.org/html/2410.11612v1
212. https://arxiv.org/pdf/2508.10574.pdf
213. https://www.thethingsnetwork.org/forum/t/fair-use-policy-explained/1300
214. https://www.bohrium.com/paper-details/federated-learning-and-tinyml-on-iot-edge-devices-challenges-advances-and-future-directions/1148657804865699889-2702
215. https://dsg.ac.upc.edu/node/974/
216. https://zenodo.org/records/15240508
217. https://dblp.org/rec/journals/corr/abs-2410-11612.html
218. https://www.themoonlight.io/ko/review/federated-learning-framework-for-lorawan-enabled-iiot-communication-a-case-study